RV College of Engineering®

UNIT-III

Go, change the world®

RV College of Engineering®

Relational Database Design- topic mapping in textbook 7th edition

- Functional Dependencies – Definition, 14.2, 14.2.1
- Inference Rules-15.1,15.1.1

- **General Equivalence of sets of FD"s,**
- **Minimal Set of FD"s ;**
- **Normal Forms Based on Primary Keys;**
- **Definitions of Second and ,Third Normal Forms;**
- **Boyce-Codd Normal Form;**
- **Properties of Relational Decompositions**

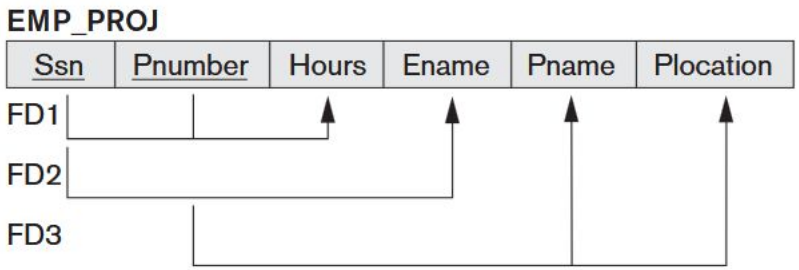
RV College of Engineering®

Definition

- formal tool for analysis of relational schemas.
- A functional dependency is a constraint between two sets of attributes from the database.
- A functional dependency, denoted by $X \rightarrow Y$, between two sets of attributes X and Y that are subsets of R specifies a constraint on the possible tuples that can form a relation state r of R. The constraint is that, for any two tuples t1 and t2 in r that have t1[X] = t2[X], they must also have t1[Y] = t2[Y].
- The set of attributes X is called the left-hand side of the FD, and Y is called the right-hand side.
- use of functional dependencies is to describe further a relation schema R by specifying constraints on its attributes that must hold at all times.
- must be defined explicitly by someone who knows the semantics of the attributes of R

RV College of Engineering®

Contd..



the value of an employee's Social Security number (Ssn) uniquely determines the employee name (Ename),
The value of a project's number (Pnumber) uniquely determines the project name (Pname) and location (Plocation), and
a combination of Ssn and Pnumber values uniquely determines the number of hours the employee currently works on the project per week (Hours).

{State, Driver_license_number} → Ssn

RV College of Engineering®

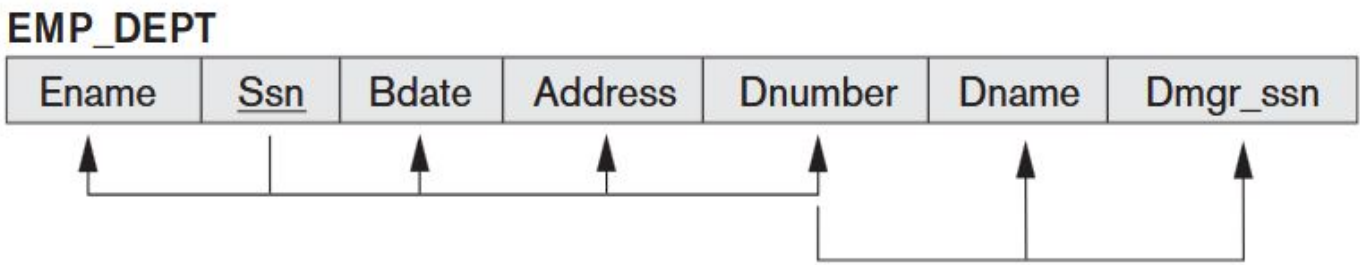
Inference rules

- the schema designer specifies the functional dependencies that are semantically obvious;
- other dependencies can be inferred or deduced from the FDs in F.
- if each department has one manager, so that Dept_no uniquely determines Mgr_ssn (Dept_no → Mgr_ssn), and
- a manager has a unique phone number called Mgr_phone (Mgr_ssn → Mgr_phone),
- then these two dependencies together imply that Dept_no → Mgr_phone.
- This is an inferred FD and need not be explicitly stated in addition to the two given FDs.
- a concept called closure formally includes all possible dependencies that can be inferred from the given set F, denoted by F^+

RV College of Engineering®

Closure

- Formally, the set of all dependencies that include F as well as all dependencies that can be inferred from F is called the closure of F; it is denoted by F+



F = {Ssn → {Ename, Bdate, Address, Dnumber},
Dnumber → {Dname, Dmgr_ssn} }—specified inference
Ssn → {Dname, Dmgr_ssn} --additional inference
Ssn → Ssn-- additional inference
Dnumber → Dname---additional inference that can be drawn

RV College of Engineering®

Closure

- Closure F^+ of F is the set of all functional dependencies that can be inferred from F.
- We need inference rules to infer new dependencies systematically.
- In the inference rules IR1 to IR3 are called Armstrong's Axioms.they are considered sound and complete.
- Sound means any set of Functional dependencies F specified on a relation schema R,any dependency that we infer from F using IR1 to IR3 holds in every relation state r of R.
- By Complete we mean, using IR1 to IR3 to infer dependencies repeatedly until no more dependencies can be infered results in a complete set of all possible dependencies.

RV College of Engineering®

Inference rules

IR1 (reflexive rule)[1]: If $X \supseteq Y$, then $X \rightarrow Y$.

IR2 (augmentation rule)[2]: $\{X \rightarrow Y\} \models XZ \rightarrow YZ$.

IR3 (transitive rule): $\{X \rightarrow Y, Y \rightarrow Z\} \models X \rightarrow Z$.

IR4 (decomposition, or projective, rule): $\{X \rightarrow YZ\} \models X \rightarrow Y$.

IR5 (union, or additive, rule): $\{X \rightarrow Y, X \rightarrow Z\} \models X \rightarrow YZ$.

IR6 (pseudotransitive rule): $\{X \rightarrow Y, WY \rightarrow Z\} \models WX \rightarrow Z$.

RV College of Engineering®

Inference rules

- The reflexive rule (IR1) states that a set of attributes always determines itself or any of its subsets, which is obvious.
- The augmentation rule (IR2) says that adding the same set of attributes to both the left- and right-hand sides of a dependency results in another valid dependency.
- The decomposition rule (IR4) says that we can remove attributes from the right-hand side of a dependency; applying this rule repeatedly can decom pose the FD $X \rightarrow \{A1, A2, ..., An\}$ into the set of dependencies $\{X \rightarrow A1, X \rightarrow A2, ..., X \rightarrow An\}$.
- The pseudotransitive rule (IR6) allows us to replace a set of attributes Y on the left hand side of a dependency with another set X that functionally determines Y, and can be derived from IR2 and IR3 if we augment the first functional dependency $X \rightarrow Y$ with W (the augmentation rule) and then apply the transitive rule

RV College of Engineering®

Algo 15.1/ Algo 16.1(if 6th editon)

Algorithm 16.1. Determining X^+, the Closure of X under F

Input: A set F of FDs on a relation schema R, and a set of attributes X, which is a subset of R.

$X^+ := X$;
repeat
    $oldX^+ := X^+$;
    for each functional dependency $Y \rightarrow Z$ in F do
        if $X^+ \supseteq Y$ then $X^+ := X^+ \cup Z$;
until $(X^+ = oldX^+)$;

RV College of Engineering®

Algo 15.2a/16.2a(6th edition)

Algorithm 16.2(a). Finding a Key K for R Given a set F of Functional Dependencies

Input: A relation R and a set of functional dependencies F on the attributes of R.

1. Set $K := R$.
2. For each attribute A in K

   {compute $(K - A)^+$ with respect to F;

   if $(K - A)^+$ contains all the attributes in R, then set $K := K - \{A\}$ };

RV College of Engineering®

Equivalence of Sets of Functional Dependencies

- Definition. A set of functional dependencies F is said to cover another set of functional dependencies E if every FD in E is also in F+; that is, if every depen dency in E can be inferred from F; alternatively, we can say that E is covered by F.
- Definition. Two sets of functional dependencies E and F are equivalent if E+ = F+. Therefore, equivalence means that every FD in E can be inferred from F, and every FD in F can be inferred from E; that is, E is equivalent to F if both the conditions—E covers F and F covers E—hold.

We can determine whether F covers E by calculating X+ with respect to F for each FD $X \rightarrow Y$ in E, and then checking whether this X+ includes the attributes in Y. If this is the case for every FD in E, then F covers E.

RV College of Engineering®

Minimal Sets of Functional Dependencies

- a minimal cover of a set of functional dependencies E is a set of functional dependencies F that satisfies the property that every dependency in E is in the closure F+ of F.

- Definition. A minimal cover of a set of functional dependencies E is a minimal set of dependencies (in the standard canonical form and without redundancy) that is equivalent to E. We can always find at least one minimal cover F for any set of dependencies E

RV College of Engineering®

Minimal Sets of Functional Dependencies

- formally define a set of functional dependencies F to be minimal if it satisfies the following conditions:
- 1. Every dependency in F has a single attribute for its right-hand side.
- 2. We cannot replace any dependency $X \rightarrow A$ in F with a dependency $Y \rightarrow A$, where Y is a proper subset of X, and still have a set of dependencies that is equivalent to F.
- 3. We cannot remove any dependency from F and still have a set of dependencies that is equivalent to F.

RV College of Engineering®

Algorithm 16.2. Finding a Minimal Cover F for a Set of Functional Dependencies E

- Input: A set of functional dependencies E.

1. Set F := E.
2. Replace each functional dependency $X \rightarrow \{A1, A2, ..., An\}$ in F by the n func-tional dependencies $X \rightarrow A1$, $X \rightarrow A2$, ..., $X \rightarrow An$.
3. For each functional dependency $X \rightarrow A$ in F for each attribute B that is an element of X if $\{ \{F - \{X \rightarrow A\} \} \cup \{ (X - \{B\}) \rightarrow A\} \}$ is equivalent to F then replace $X \rightarrow A$ with $(X - \{B\}) \rightarrow A$ in F.
4. For each remaining functional dependency $X \rightarrow A$ in F if $\{F - \{X \rightarrow A\} \}$ is equivalent to F, then remove $X \rightarrow A$ from F

RV College of Engineering®

Normal forms based on Primary keys

- Formal methodology for testing and improving relation schemas.
- Proposed by Codd in 1972., given a set of functional dependencies along with a primary key combined with tests for normal forms drives the normalization process.
- Takes a relation through a series of tests to certify whether it satisfies a certain normal forms.
- Takes top down approach
- Evaluates each relation against the criteria for normal forms and decompose relations as necessary.
- Codd proposed three normal forms :
- 1NF,2NF,3NF(also called BCNF)
- All the three normal forms are based on functional dependencies among the attributes of a relation.

RV College of Engineering®

Normal forms based on Primary keys

- 4NF, 5NF are based on the concept of multivalued dependencies and join dependencies,
- Normalization of data can be considered a process of analyzing the given relation schemas based on their FDs and primary keys to achieve the desirable properties of
- (1) minimizing redundancy and
- (2) minimizing the insertion, deletion, and update anomalies.
- It can be considered as a “filtering” or “purification” process to make the design have successively better quality.

RV College of Engineering®

Normal forms based on Primary keys

- the normalization procedure provides database designers with the following:

■ A formal framework for analyzing relation schemas based on their keys and on the functional dependencies among their attributes

■ A series of normal form tests that can be carried out on individual relation schemas so that the relational database can be normalized to any desired degree

RV College of Engineering®

First Normal Form(1NF)

- states that the domain of an attribute must include only atomic (simple, indivisible) values, the value of any attribute in a tuple must be a single value from the domain of that attribute.
- 1NF disallows having a set of values, a tuple of values, or a combination of both as an attribute value for a single tuple.

Dname	Dnum	dlocation
Research	4	Bang, mysore
Administration	5	Bang
Headquarters	2	chennai

RV College of Engineering®

Three methods to reduce to 1NF

- Place redundant attribute along with primary key in separate table.
- Expand the key which is having multiple values and write it in separate tuples
- If upper limit of the multivalued attribute is known then create multiple columns for that attribute

- First normal form also disallows multivalued attributes that are themselves composite. These are called nested relations because each tuple can have a relation within it.

EMP_PROJ(Ssn, Ename, {PROJS(Pnumber, Hours)})

RV College of Engineering®

1NF

EMP_PROJ1

Ssn	Ename

EMP_PROJ2

Ssn	Pnumber	Hours

RV College of Engineering®

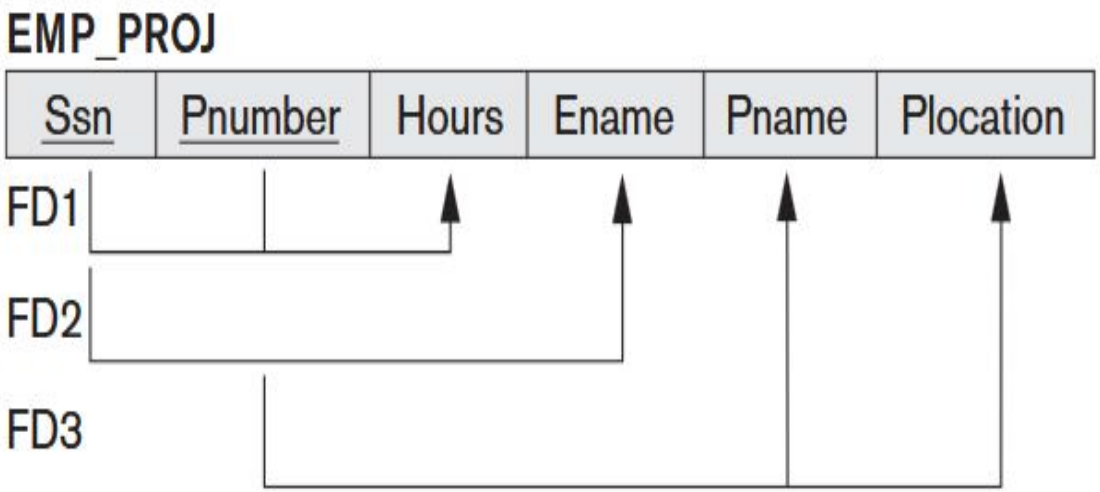
Second Normal Form(2NF)

- A table is said to be in 2NF if it is in 1NF and there is no partial dependency.
- A relation schema R is in 2NF if every nonprime attribute A in R is fully functionally dependent on the primary key of R.
- {Ssn, Pnumber} → Hours is a full dependency (neither Ssn → Hours nor Pnumber → Hours holds).

However, the dependency {Ssn, Pnumber} → Ename is partial because Ssn → Ename holds.

RV College of Engineering®

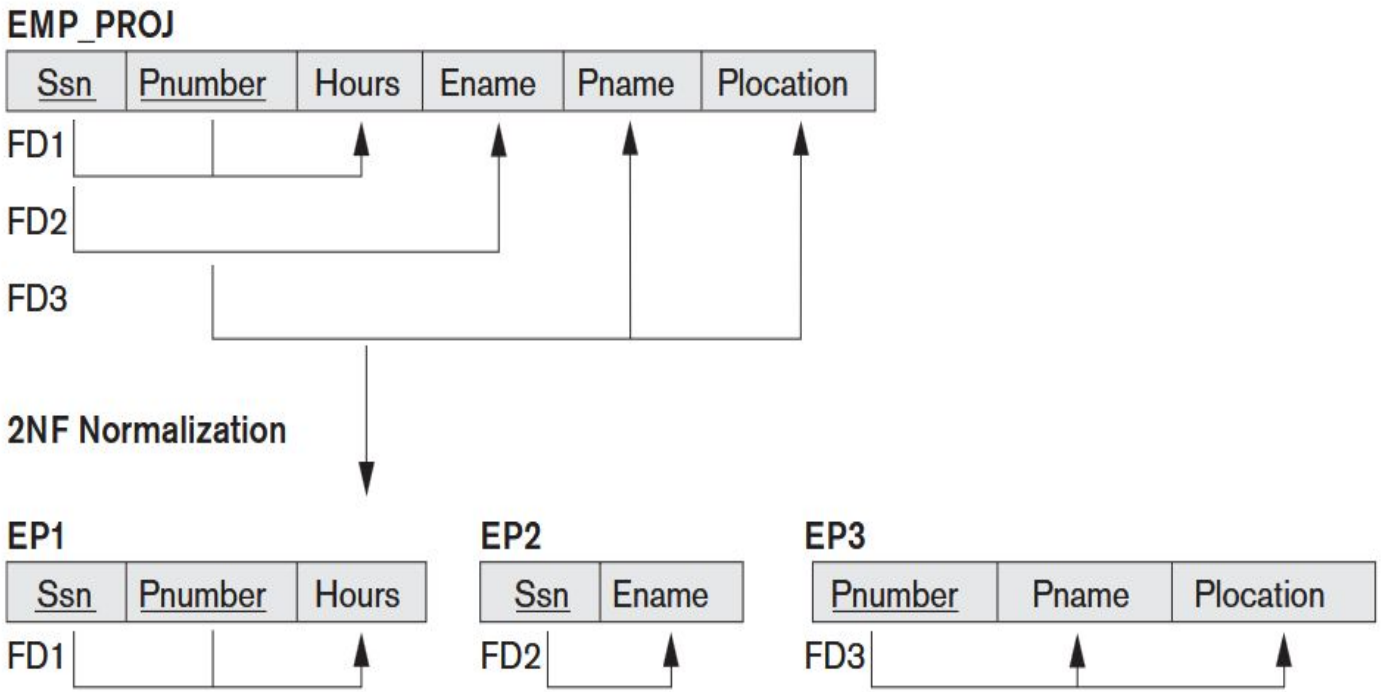
Second Normal Form(2NF)



nonprime attributes are associated only with the part of the primary key on which they are fully functionally dependent.

RV College of Engineering®

Example



RV College of Engineering®

Example

- 1NF: PERSON (Ss#, {Car_lic#}, {Phone#})
- CANDIDATE(Ssn,Name,{JOB_HIST(Company,Highest_position,{SAL_HIST(Year,Max_sal)})})

RV College of Engineering®

Third normal form(3NF)

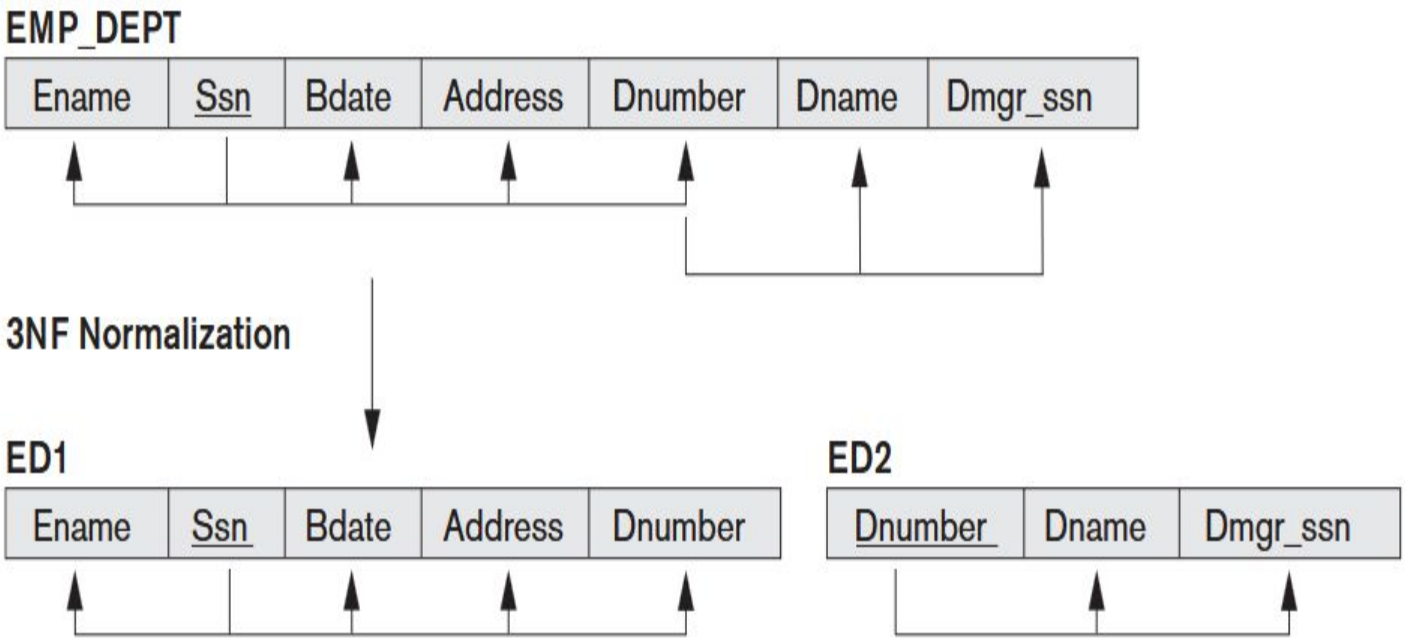
- A table is said to be in 3NF , if it is in 2NF and if there is no transitive dependency for non-prime attributes, then the table is in 3NF.
- A functional dependency X → Y in a relation schema R is a transitive dependency if there exists a set of attributes Z in R that is neither a candidate key nor a subset of any key of R, 10 and both X → Z and Z → Y hold.
- The dependency Ssn → Dmgr_ssn is transitive through Dnumber in EMP_DEPT because both the dependencies Ssn → Dnumber and

Dnumber → Dmgr_ssn hold and Dnumber is nei ther a key itself nor a subset of the key of EMP_DEPT.

- According to Codd's original definition, a relation schema R is in 3NF if it satisfies 2NF and no nonprime attribute of R is transitively dependent on the primary key.

RV College of Engineering®

Third normal form(3NF)



RV College of Engineering®

BCNF(Boyce-Codd Normal Form)

- every relation in BCNF is also in 3NF.
- A relation schema R is in BCNF if whenever a nontrivial functional dependency X → A holds in R, then X is a superkey of R.
- It enforces that every non-trivial functional dependency must have a superkey on its left-hand side.

- **Rules for BCNF**
- ***Rule 1:*** *The table should be in the 3rd Normal Form.*
- ***Rule 2:*** *X should be a super-key for every functional dependency (FD) X−>Y in a given relation.*

RV College of Engineering®

Example

- .

Student	Teacher	Subject
Aaa	Navathe	Dbms
Aaa	Balguruswamy	C
Bbb	Navathe	Dbms
bbb	Dennis ritchie	c

```
FD: { (student, Teacher) -> subject, (student, subject) -> Teacher, (Teacher) -> subject}
```

RV College of Engineering®

Example

- The above relation is not in BCNF, because in the FD (teacher->subject), teacher is not a key.
- If the student AAA is deleted then we will loose information balguruswamy teaches c, because the teacher is a determinant but not a candidate key.
- Divide into two Relations : Teacher-> subject , Student-> Teacher

RV College of Engineering®

Example

Stu_ID	Stu_Branch	Stu_Course	Branch_Number	Stu_Course_No
101	Computer Science & Engineering	DBMS	B_001	201
101	Computer Science & Engineering	Computer Networks	B_001	202
102	Electronics & Communication Engineering	VLSI Technology	B_003	401
102	Electronics & Communication Engineering	Mobile Communication	B_003	402

RV College of Engineering®

- Functional dependency for the above tables are:
- Stu_ID −> Stu_Branch
  Stu_Course −> {Branch_Number, Stu_Course_No}

- Candidate Keys of the above table are: **{Stu_ID, Stu_Course}**

- For satisfying this table in BCNF, we have to decompose it into further tables.
- **Stu_Branch Table:**

Stu_ID	Stu_Branch

- **Stu_Course Table:**

Stu_Course	Branch_Number	Stu_Course_No

- **Stu_Enroll Table:**

Stu_ID	Stu_Course_No

RV College of Engineering®

Properties of Relational Decompositions

- Relation Decomposition and Insufficiency of Normal Forms
- Dependency Preservation and Property of a Decomposition
- Nonadditive (Lossless) Join Property of a Decomposition
- Testing Binary Decompositions for the Nonadditive Join Property
- Successive Nonadditive Join Decompositions

RV College of Engineering®

Relation Decomposition and Insufficiency of Normal Forms

- universal relation assumption, which states that every attribute name is unique.
- The set F of functional dependencies that should hold on the attributes of R is specified by the database designers and is made available to the design algorithms.
- Using the functional dependencies, the algorithms decompose the universal relation schema R into a set of relation schemas D= {R1 , R2 , ..., R m} that will become the relational database schema; D is called a **decomposition** of R.
- make sure that each attribute in R will appear in at least one relation schema
- This is called the **attribute preservation** condition of a decomposition.
- Another goal is to have each individual relation R i in the decomposition D be in BCNF or 3NF

RV College of Engineering®

Dependency Preservation and Property of a Decomposition

- It would be useful if each functional dependency X→Y specified in F either appeared directly in one of the relation schemas Ri in the decomposition D or could be inferred from the dependencies that appear in some R i. Informally, this is the dependency preservation condition.

Definition. Given a set of dependencies F on R, the **projection** of F on R_i, denoted by $\pi_{Ri}(F)$ where R_i is a subset of R, is the set of dependencies $X \rightarrow Y$ in F^+ such that the attributes in $X \cup Y$ are all contained in R_i. Hence, the projection of F on each relation schema R_i in the decomposition D is the set of functional dependencies in F^+, the closure of F, such that all their left- and right-hand-side attributes are in R_i. We say that a decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R is **dependency-preserving** with respect to F if the union of the projections of F on each R_i in D is equivalent to F; that is, $((\pi_{R_1}(F)) \cup \ldots \cup (\pi_{R_m}(F)))^+ = F^+$.

RV College of Engineering®

Nonadditive (Lossless) Join Property of a Decomposition

- nonadditive join property,ensures that no spurious tuples are generated when a NATURAL JOIN operation is applied to the relations resulting from the decomposition.

Definition. Formally, a decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R has the **lossless (nonadditive) join property** with respect to the set of dependencies F on R if, for *every* relation state r of R that satisfies F, the following holds, where $*$ is the NATURAL JOIN of all the relations in D: $*(\pi_{R_1}(r), \ldots, \pi_{R_m}(r)) = r$.

RV College of Engineering®

Book_ISBN	Price	Pagecount
9	25	800
1	22	200
2	25	810

Decomposition A

ISBN	PRICE
9	25
1	22
2	25

Isbn	Page count
9	800
1	200
2	810

Decomposition B

ISBN	PRICE
9	25
1	22
2	25

Pagecount	Price
800	25
200	22
810	25

RV College of Engineering®

Performing natural join on decompositions A and B

ISBN	Price	Paggecount
9	25	800
1	22	200
2	25	810

ISBN	Price	Pagecount
9	25	800
9	25	810
2	25	800
2	25	810
1	22	200

RV College of Engineering®

Example:

- Consider the schema R(ABCDE) with the FDs A->B, C->D. Check if the following decomposition of R are
- i. Dependency Preserving
- Ii Lossless join
- R1(ACE), R2(CD) snd R3(AB) are the relations given :

- To check property 1 Check the relation against FDs to check whether FDs are existing In the relations.
- R1- has nothing
- R2- has F1=C->D
- R3 has F2=A->B
- hence F1UF2=F(Original FDs)

RV College of Engineering®

II. Checking for lossless decomposition

- Create a matrix with attributes of original relation as columns and relation as rows , Fill a and b in the matrix based on the attributes present in the relation , if an attribute is present then mark the corresponding row as a otherwise b.

	A	B	C	D	E
R1	a	b	a	b	a
R2	b	b	a	a	b
R3	a	a	b	b	b

RV College of Engineering®

II. Checking for lossless decomposition

- Now next step , take up functional dependency one by one ,
- A->B= if there is atleast 2 rows with value a on the left hand side then change the right hand side to a :
- Conclusion: if none of the rows have all a's then it's a lossy decomposition

	A	B	C	D	E
R1	a	**b,a**	a	**b,a**	a
R2	b	b	a	a	b
R3	a	a	b	b	b

RV College of Engineering®

Example 2:

- Consider the relation schema R(ssn,ename, pnum,pname,ploc,hours) as the decompositon of R as follows
- R1(ssn,ename)
- R2(pno,pname,ploc)
- R3(ssn,pnum,hours)

- The following FDs holds on R
- Ssn->ename,pnum
- Pnum->pname,ploc
- Ssn,pnum->hours

- Determine if the decomposition is lossless or nor

RV College of Engineering®

	Ssn	Ename	Pnum	pname	Ploc	hours
R1	a	a	b	b	b	b
R2	b	b	a	a	a	b
R3	a	b	a	b	b	a

RV College of Engineering®

- Change the values of relation from b to a if the condition satisfies :

	Ssn	Ename	Pnum	pname	Ploc	hours
R1	a	a	B,a	B,a	B,a	B,a
R2	b	B,a	a	a	a	b
R3	a	b	a	b	b	a

RV College of Engineering®

Example

- Sp(sno,sname, scity, status,pnum,pname,price, qty)

- Decompositions are : cs(scity, status),supp(sno,sname,scity), Part(pno,pname,price),spx(sno,pnum,qty)

- FDs on SP are
- Sno-> sname,city
- Scity->status
- Pnum->pname,price
- Sno,pnum->qty

Algorithm 16.3. Testing for Nonadditive Join Property

Input: A universal relation R, a decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R, and a set F of functional dependencies.

Note: Explanatory comments are given at the end of some of the steps. They follow the format: (* *comment* *).

1. Create an initial matrix S with one row i for each relation R_i in D, and one column j for each attribute A_j in R.
2. Set $S(i, j) := b_{ij}$ for all matrix entries. (* each b_{ij} is a distinct symbol associated with indices (i, j) *).
3. For each row i representing relation schema R_i
   {for each column j representing attribute A_j
   {if (relation R_i includes attribute A_j) then set $S(i, j) := a_j$;};}; (* each a_j is a distinct symbol associated with index (j) *).

4. Repeat the following loop until a *complete loop execution* results in no changes to S

   {for each functional dependency $X \rightarrow Y$ in F

   {for all rows in S *that have the same symbols* in the columns corresponding to attributes in X

   {make the symbols in each column that correspond to an attribute in Y be the same in all these rows as follows: If any of the rows has an *a* symbol for the column, set the other rows to that *same a* symbol in the column. If no *a* symbol exists for the attribute in any of the rows, choose one of the *b* symbols that appears in one of the rows for the attribute and set the other rows to that same *b* symbol in the column ;} ; } ;};

5. If a row is made up entirely of *a* symbols, then the decomposition has the nonadditive join property; otherwise, it does not.

RV College of Engineering®

Testing Binary Decompositions for the Nonadditive Join Property

- A decomposition D = {R1, R2} of R has the lossless (nonadditive) join property with respect to a set of functional dependencies F on R if and only if either
- The FD $((R1 \cap R2) \rightarrow (R1 - R2))$ is in F+, or
- The FD $((R1 \cap R2) \rightarrow (R2 - R1))$ is in F+

RV College of Engineering®

Example

- Consider the relation R(ABC) with FD A->B and B->C . Find the following decompostioin of R is a lossless decomposition
- R1(AC) and R2(AB)

- Given A->B,B->C, A->C(transitivity)
- Decompositions are R1(AC), R2(AB)
- Dependency A->C exists in R1 and Dependency A->B exists in R2 but Dependency B->C is not existing hence Dependency preservation is not done

- R1∩ $R2$=A
- R1-R2=C
- R2-R1=B

RV College of Engineering®

- Check $((R1 \cap R2) \rightarrow (R1 - R2))$
- A->C

- Check $((R1 \cap R2) \rightarrow (R2 - R1))$
- A->B

- One of the decomposition holds hence it is lossless

RV College of Engineering®

UNIT-IV

- **Transaction Processing Concepts-** Introduction to transaction processing, Transaction states and additional operations, Desirable properties of transaction, Schedules of transactions. Characterizing schedules based on Serializability: Serial, Non serial and Conflict- Serializable schedules, Testing for Conflict serializability of schedule.

- **Concurrency Control Techniques:** Two phase locking techniques for concurrency control, types of locks and system lock tables.

RV College of Engineering®

Objective

- basic concepts and theory that are needed to ensure the correct executions of transactions.
- why concurrency control and recovery are necessary
- ACID properties

RV College of Engineering®

Introduction

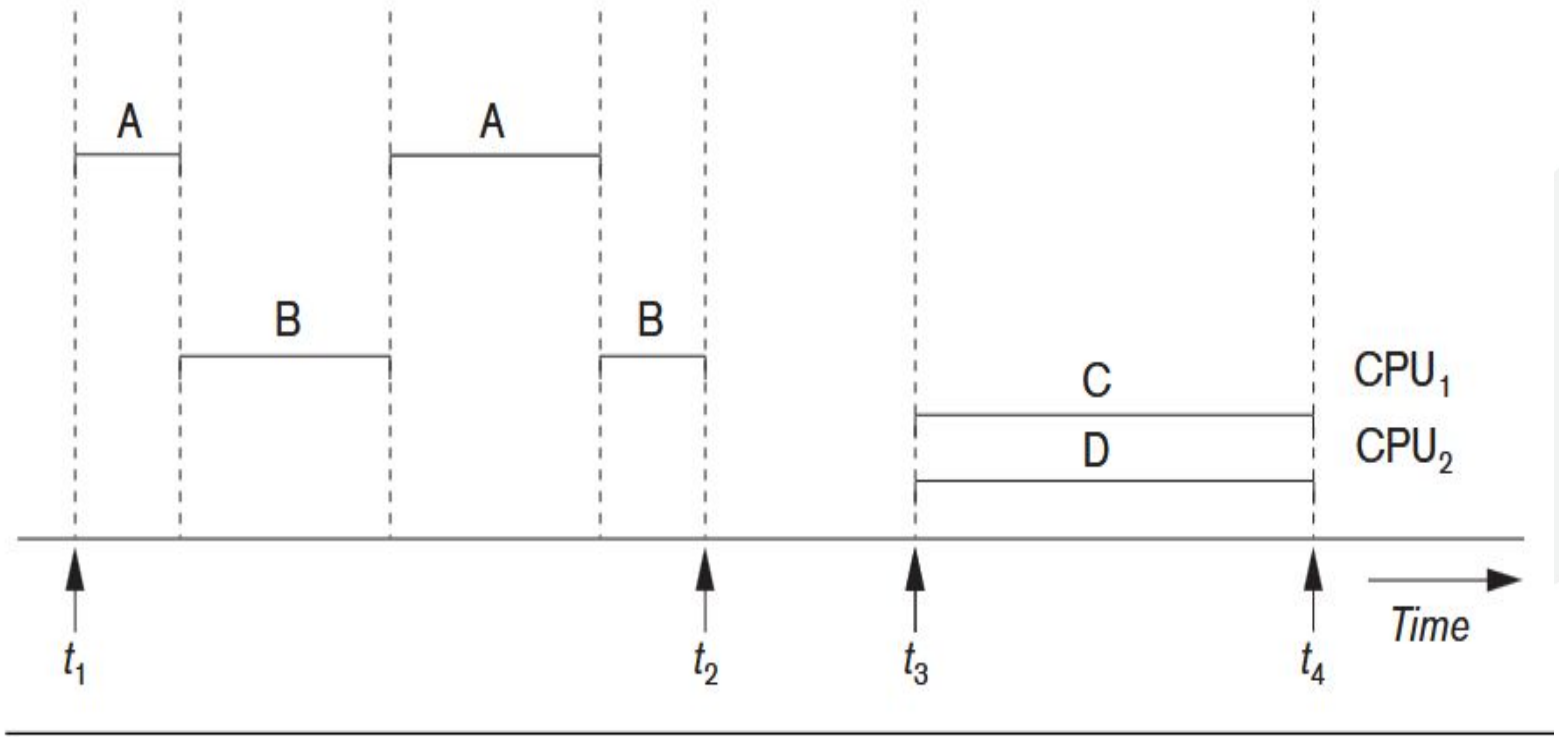
- Transaction processing systems are systems with large databases and hundreds of concurrent users executing database transactions.
- concept of transaction provides a mechanism for describing logical units of database processing.
- Ex: airline reservations, banking, credit card processing, online retail purchasing, stock markets, supermarket checkouts.
- A transaction is typically implemented by a computer program, which includes database commands such as retrievals, insertions, deletions, and updates.

RV College of Engineering®

- Single-User versus Multiuser Systems
- Transactions, Database Items, Read and Write Operations, and DBMS Buffers
- Why Concurrency Control Is Needed
- Why Recovery is needed

RV College of Engineering®

Single-User versus Multiuser Systems



RV College of Engineering®

Transactions, Database Items, Read and Write Operations, and DBMS Buffers

- The database operations that form a transaction can either be embedded within an application program or they can be specified interactively via a high-level query language such as SQL.
- specifying the transaction boundaries is by specifying **explicit begin** transaction and **end transaction** statements in an application program
- Two types of transactions:
- Read only
- Read write
- read_item(X). Reads a database item named X into a program variable.
- write_item(X). Writes the value of program variable X into the database
- item named X.

RV College of Engineering®

Sequence of operations for read and write

- executing a read_item(X) command includes the following
- steps:
- 1. Find the address of the disk block that contains item X.
- 2. Copy that disk block into a buffer in main memory (if that disk block is not
- already in some main memory buffer).
- 3. Copy item X from the buffer to the program variable named X.

- Executing a write_item(X) command includes the following steps:
- 1. Find the address of the disk block that contains item X.
- 2. Copy that disk block into a buffer in main memory (if that disk block is not already in some main memory buffer).
- 3. Copy item X from the program variable named X into its correct location in the buffer.
- 4. Store the updated block from the buffer back to disk (either immediately or at some later point in time).

RV College of Engineering®

Why Concurrency Control Is Needed

- Concurrency control and recovery mechanisms are mainly concerned with the database commands in a transaction.
- Transactions submitted by the various users may execute concurrently and may access and update database.
- Consider the transaction below:

(b)

T_1	T_2
read_item(X);	read_item(X);
$X := X - N$;	$X := X + M$;
write_item(X);	write_item(X);
read_item(Y);	
$Y := Y + N$;	
write_item(Y);	

RV College of Engineering®

Problems if concurrency is not controlled

- The Lost Update Problem.
- The Temporary Update (or Dirty Read) Problem.
- The Incorrect Summary Problem.
- The Unrepeatable Read Problem.

RV College of Engineering®

Lost update problem

This problem occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of some database items incorrect.

	T_1	T_2
Time ↓	read_item(X);	
	$X := X - N$;	
		read_item(X);
		$X := X + M$;
	write_item(X);	
	read_item(Y);	
		write_item(X);
	$Y := Y + N$;	
	write_item(Y);	

RV College of Engineering®

The Temporary Update (or Dirty Read) Problem.

- This problem occurs when one transaction updates a database item and then the transaction fails for some reason

Time ↓

T_1	T_2
read_item(X);	
$X := X - N$;	
write_item(X);	
	read_item(X);
	$X := X + M$;
	write_item(X);
read_item(Y);	

RV College of Engineering®

The Incorrect Summary Problem

- If one transaction is calculating an aggregate summary function on a number of database items while other transactions are updating some of these items, the aggregate function may calculate some values before they are updated and others after they are updated.

T_1	T_3

	sum := 0;
	read_item(A);
	sum := sum + A;
	⋮

read_item(X);
X := X − N;
write_item(X);	

	read_item(X);
	sum := sum + X;
	read_item(Y);
	sum := sum + Y;

read_item(Y);
Y := Y + N;
write_item(Y);	

RV College of Engineering®

The Unrepeatable Read Problem.

- Transaction T reads the same item twice and the item is changed by another transaction T between the two reads.

RV College of Engineering®

Why Recovery Is Needed

- Whenever a transaction is submitted to a DBMS for execution either it must be:
- **committed**
- **aborted**

- **Failures are generally classified as transaction, system, and media failures.**

RV College of Engineering®

reasons for a transaction to fail in the middle of execution:

- **A computer failure (system crash): A hardware, software, or network error occurs in the computer system during transaction execution , media failure.**
- **A transaction or system error: Some operation in the transaction may cause it to fail, such as integer overflow or division by zero, logical**
- **programming error, interrupt.**
- **Local errors or exception conditions detected by the transaction: During transaction execution, certain conditions may occur that necessitate cancellation of the transaction. ex: insufficient balance in account**

RV College of Engineering®

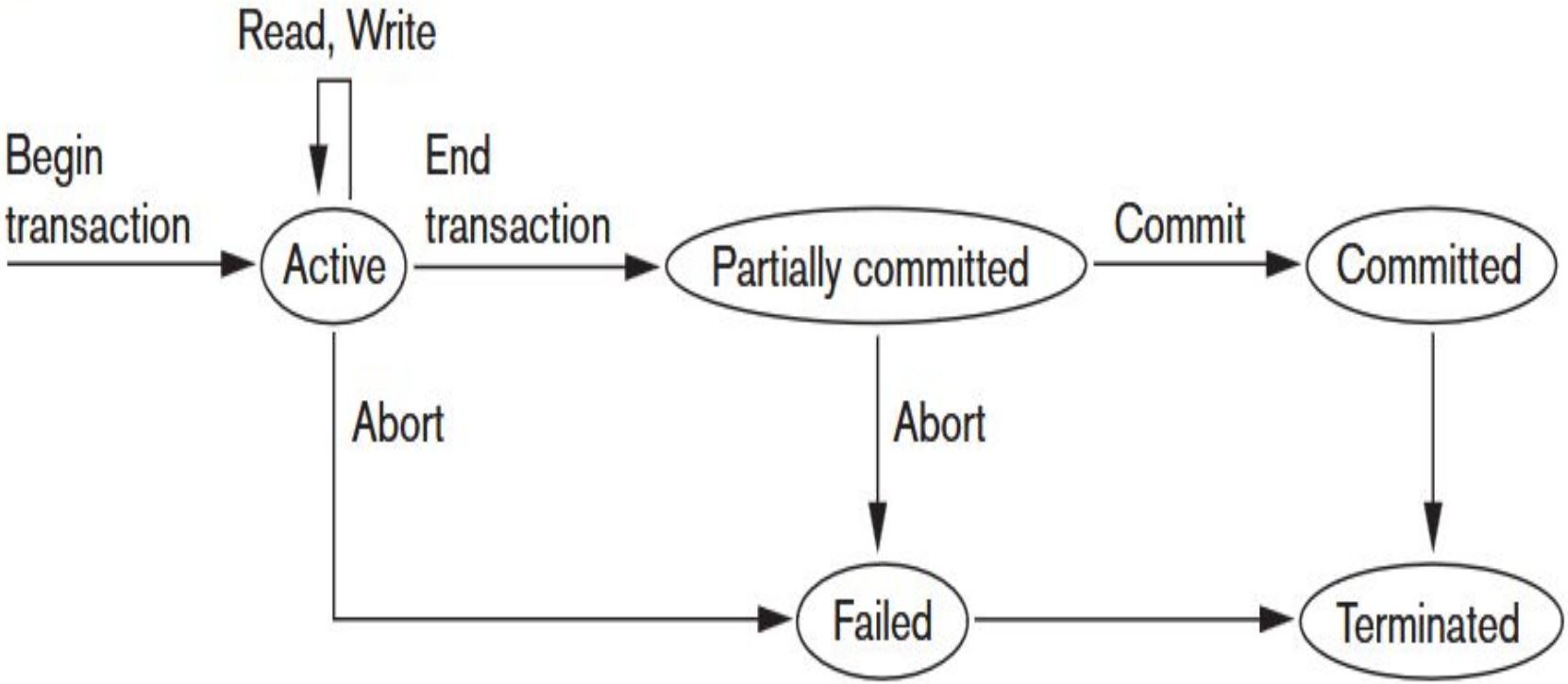
reasons for a transaction to fail in the middle of execution:

- **Concurrency control enforcement: The concurrency control method may decide to abort a transaction because it violates serializability.**
- **Disk failure:Some disk blocks may lose their data because of a read or write malfunction or because of a disk read/write head crash**
- **Physical problems and catastrophes.**

RV College of Engineering®

Transaction States and Additional Operations

- recovery manager of the DBMS needs to keep track of the following operations:
- BEGIN_TRANSACTION. This marks the beginning of transaction execution.
- READ or WRITE. These specify read or write operations on the database items that are executed as part of a transaction.
- END_TRANSACTION. This specifies that READ and WRITE transaction operations have ended and marks the end of transaction execution.
- COMMIT_TRANSACTION. This signals a successful end of the transaction so that any changes (updates) executed by the transaction can be safely committed to the database and will not be undone.
- ROLLBACK (or ABORT). This signals that the transaction has ended unsuc-cessfully, so that any changes or effects that the transaction may have applied to the database must be undone.


Read, Write
Begin transaction
Active
End transaction
Partially committed
Commit
Committed
Abort
Abort
Failed
Terminated

RV College of Engineering®

The System Log

- The log is a sequential, append-only file that is kept on disk, so it is not affected by any type of failure except for disk or catastrophic failure.

RV College of Engineering®

Desirable Properties of Transactions

- Atomicity.
- Consistency preservation.
- Isolation.
- Durability or permanency.

RV College of Engineering®

Characterizing schedules based on serializability:serial schedules

Time ↓

T_1	T_2
read_item(X);	
$X := X - N$;	
write_item(X);	
read_item(Y);	
$Y := Y + N$;	
write_item(Y);	
	read_item(X);
	$X := X + M$;
	write_item(X);

Schedule A

(b)

Time ↓

T_1	T_2
	read_item(X);
	$X := X + M$;
	write_item(X);
read_item(X);	
$X := X - N$;	
write_item(X);	
read_item(Y);	
$Y := Y + N$;	
write_item(Y);	

Schedule B

RV College of Engineering®

Non serial schedules

ne

T_1	T_2
read_item(X);	
$X := X - N$;	
	read_item(X);
	$X := X + M$;
write_item(X);	
read_item(Y);	
	write_item(X);
$Y := Y + N$;	
write_item(Y);	

Schedule C

Time

T_1	T_2
read_item(X);	
$X := X - N$;	
write_item(X);	
	read_item(X);
	$X := X + M$;
	write_item(X);
read_item(Y);	
$Y := Y + N$;	
write_item(Y);	

Schedule D

RV College of Engineering®

Serial, Non serial and Conflict- Serializable schedules

- a schedule S is serial if, for every transaction T participating in the schedule, all the operations of T are executed consecutively in the schedule; otherwise, the schedule is called nonserial.
- The problem with serial schedules is that they limit concurrency by prohibiting interleaving of operations

The definition of serializable schedule is as follows: A schedule S of n transactions is serializable if it is equivalent to some serial schedule of the same n transactions.
there are n! possible serial schedules of n transactions and many more possible nonserial schedules.

RV College of Engineering®

Serial, Non serial and Conflict- Serializable schedules

- ways to define schedule equivalence:comparing the effects of the schedules on the database. Two schedules are called result equivalent if they produce the same final state of the database.
- Below transaction shows same effect but they are not equivalent

S_1	S_2

read_item(X);
$X := X + 10$;
write_item(X);
	read_item(X);
	$X := X * 1.1$;
	write_item (X);

- Two definitions of equivalence of schedules are generally used: conflict **equivalence and view equivalence**.

RV College of Engineering®

Serial, Non serial and Conflict- Serializable schedules

- Two schedules are said to be conflict equivalent if the order of any two conflicting operations is the same in both schedules.
- a schedule S to be conflict serializable if it is (conflict) equivalent to some serial schedule S.

(a)

T_1	T_2
read_item(X);	
$X := X - N$;	
write_item(X);	
read_item(Y);	
$Y := Y + N$;	
write_item(Y);	
	read_item(X);
	$X := X + M$;
	write_item(X);

Time

Schedule A

T_1	T_2
read_item(X);	
$X := X - N$;	
write_item(X);	
	read_item(X);
	$X := X + M$;
	write_item(X);
read_item(Y);	
$Y := Y + N$;	
write_item(Y);	

Time

Schedule D

RV College of Engineering®

Testing for Conflict Serializability of a Schedule

- The algorithm looks at only the read_item and write_item operations in a schedule to construct a precedence graph (or serialization graph), which is a directed graph G = (N, E) that consists of a set of nodes N = {T1, T2, ..., Tn } and a set of directed edges E = {e1 , e2, ..., em }.
- There is one node in the graph for each transaction Ti in the schedule.
- Each edge ei in the graph is of the form (Tj → Tk), 1 ≤ j ≤ n, 1 ≤ k f n, where Tj is the starting node of ei and Tk is the ending node of ei.
- Such an edge from node Tj to node T k is created by the algorithm if one of the operations in T j appears in the schedule before some conflicting operation in Tk

Algorithm 21.1. Testing Conflict Serializability of a Schedule S

1. For each transaction T_i participating in schedule S, create a node labeled T_i in the precedence graph.
2. For each case in S where T_j executes a read_item(X) after T_i executes a write_item(X), create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
3. For each case in S where T_j executes a write_item(X) after T_i executes a read_item(X), create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
4. For each case in S where T_j executes a write_item(X) after T_i executes a write_item(X), create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
5. The schedule S is serializable if and only if the precedence graph has no cycles.

RV College of Engineering®

Concurrency Control Techniques

- Locking protocols
- timestamps.

RV College of Engineering®

Two phase locking techniques for concurrency control

- A lock is a variable associated with a data item that describes the status of the item with respect to possible operations that can be applied to it.
- binary locks:A binary lock can have two states or values: locked and unlocked (or 1 and 0). lock(X) denotes the current value of lock. Two operations, lock_item and unlock_item, are used with binary locking.
- shared/exclusive locks—also known as read/write locks

RV College of Engineering®

Binary Lock

- A transaction requests access to an item X by first issuing a lock_item(X) operation.
- If LOCK(X) =1, the transaction is forced to wait. If LOCK(X) = 0, it is set to 1 (the transaction locks the item) and the transaction is allowed to access item X. lock definition is as given below and offers mutual exclusion

```
lock_item(X):
B:  if LOCK(X) = 0              (* item is unlocked *)
        then LOCK(X) ←1         (* lock the item *)
    else
        begin
        wait (until LOCK(X) = 0
            and the lock manager wakes up the transaction);
        go to B
        end;
unlock_item(X):
    LOCK(X) ← 0;                (* unlock the item *)
    if any transactions are waiting
        then wakeup one of the waiting transactions;
```

RV College of Engineering®

Binary Lock

- each lock can be a record with three fields: <Data_item_name, LOCK, Locking_transaction> plus a queue for transactions that are waiting to access the item.
- system needs to maintain only the records for the items that are currently locked in a lock table, which could be organized as a hash file on the item name.
- The DBMS has a lock manager sub-system to keep track of and control access to locks

RV College of Engineering®

Binary locking scheme

- every transaction must obey the following rules:

1. A transaction T must issue the operation lock_item(X) before any read_item(X) or write_item(X) operations are performed in T.
2. A transaction T must issue the operation unlock_item(X) after all read_item(X) and write_item(X) operations are completed in T.
3. A transaction T will not issue a lock_item(X) operation if it already holds the lock on item X. 1
4. A transaction T will not issue an unlock_item(X) operation unless it already holds the lock on item X

RV College of Engineering®

Shared/Exclusive (or Read/Write) Locks.

- multiple-mode lock is used. In this scheme—called shared/exclusive or read/write locks—there are three locking operations: read_lock(X), write_lock(X), and unlock(X).
- A lock associated with an item X, LOCK(X), now has three possible states:
- read-locked, write-locked, or unlocked.
- Each record in the lock table will have four fields: <Data_item_name, LOCK, No_of_reads, Locking_transaction(s)>.

RV College of Engineering®

Lock conditions

- When we use the shared/exclusive locking scheme, the system must enforce the following rules:

1. A transaction T must issue the operation read_lock(X) or write_lock(X) before any read_item(X) operation is performed in T.
2. A transaction T must issue the operation write_lock(X) before any write_item(X) operation is performed in T.
3. A transaction T must issue the operation unlock(X) after all read_item(X) and write_item(X) operations are completed in T
4. A transaction T will not issue a read_lock(X) operation if it already holds a read (shared) lock or a write (exclusive) lock on item X. This rule may be relaxed, as we discuss shortly.
5. A transaction T will not issue a write_lock(X) operation if it already holds a read (shared) lock or write (exclusive) lock on item X. This rule may also be relaxed, as we discuss shortly.
6. A transaction T will not issue an unlock(X) operation unless it already holds a read (shared) lock or a write (exclusive) lock on item X.

RV College of Engineering®

Read lock

```
read_lock(X):
B:  if LOCK(X) = "unlocked"
        then begin LOCK(X) ← "read-locked";
                no_of_reads(X) ← 1
                end
    else if LOCK(X) = "read-locked"
        then no_of_reads(X) ← no_of_reads(X) + 1
    else begin
            wait (until LOCK(X) = "unlocked"
                and the lock manager wakes up the transaction);
            go to B
            end;
```